



Security Audit

SafeHaven Exchange (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	33
Conclusion	34
Our Methodology	35
Disclaimers	37
About Hashlock	38

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The SafeHaven Exchange team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

SafeHaven Exchange is a blockchain-based financial platform focused on the tokenization of traditional assets like stocks, ETFs, and mutual funds. Its native token, SHE, represents utility in the exchange and is used for governance, platform access, and rewards. Built on a proprietary, AI-powered infrastructure, SafeHaven prioritizes security, regulatory compliance (KYC/AML), and transparency. The project roadmap includes launching the SafeHaven Blockchain (SHB) in 2025 and expanding its range of tokenized securities. Additionally, SafeHaven commits 1% of its future net profits to charitable initiatives through its SafeHaven Initiative program.

Project Name: SafeHaven Exchange

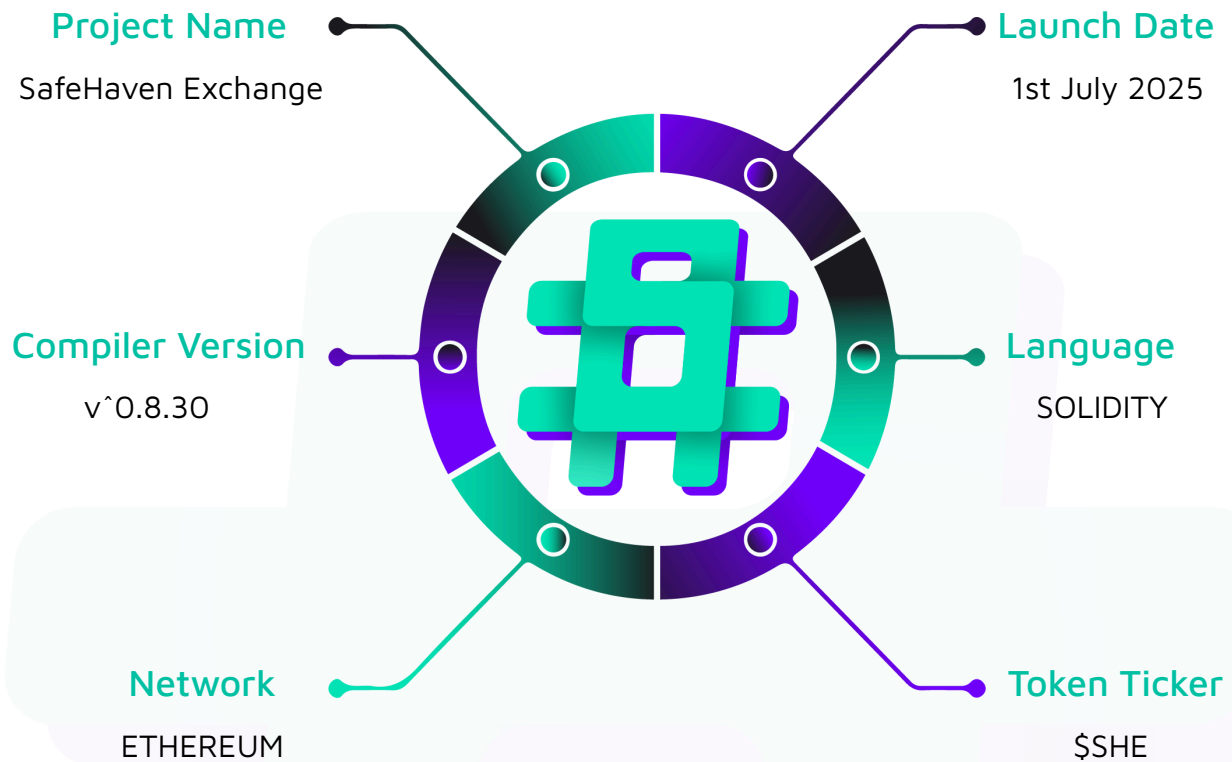
Project Type: Token

Compiler Version: ^0.8.30

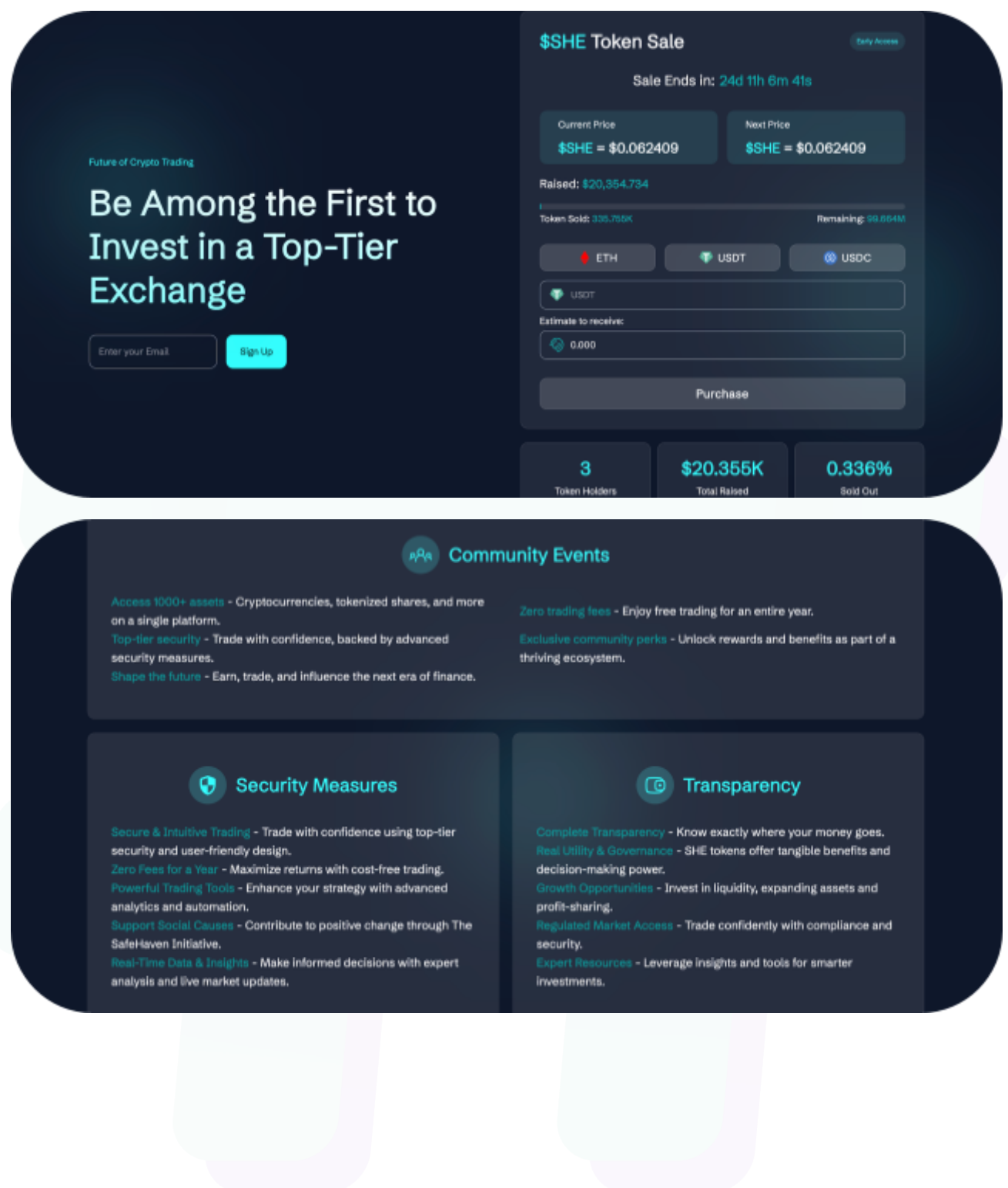
Website: <http://www.she.io>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the SafeHaven Exchange project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	SafeHaven Exchange Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract 1	SheToken.sol
Contract 2	SheVestingContract.sol
Contract 3	SheSaleContract.sol
Contract 4	ShePrivateSale.sol
Contract 5	SheAllocation.sol
Audited GitHub Commit Hash	a54f715930a6c9bc1954cff790c32f59083a2f1d
Fix Review GitHub Commit Hash	603a60196fea8b03bfcf1a74edf4312381b48cfe

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High severity vulnerabilities

4 Medium severity vulnerabilities

2 Low severity vulnerabilities

3 Gas Optimisations

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SheToken.sol - Mint and manage 1 billion SHE tokens with 18 decimals as the primary utility token for the SafeHaven ecosystem - Implement blacklisting mechanism to prevent specific addresses from transferring tokens for compliance and security purposes - Recover accidentally sent foreign ERC-20 tokens and ETH to the contract address, preventing permanent loss of funds	Contract achieves this functionality.
SheVestingContract.sol - Create and manage multiple vesting schedules with configurable cliff periods and durations - Calculate and release vested tokens based on a linear vesting formula over time - Support batch operations for adding multiple vesting schedules simultaneously - Implement controller-based access control for authorized contracts to create vesting schedules - Track released amounts to prevent double-spending and ensure accurate token distribution	Contract achieves this functionality.
SheSaleContract.sol - Manage SHE token sales with support for ETH, USDT, and USDC payment methods - Implement a dynamic pricing algorithm based on the total tokens sold with a bonding curve	Contract achieves this functionality.

formula - Integrate Chainlink price feeds for real-time ETH/USD conversion rates - Transfer purchased tokens to the vesting contract with a 90-day linear vesting schedule - Enforce purchase limits, cooldown periods, and sale duration constraints to prevent abuse - Enable owner withdrawal of collected funds and remaining tokens after sale completion	
ShePrivateSale.sol - Manage private token sales for early investors with immediate and vested token distribution - Distribute 25% of allocated tokens immediately and vest the remaining 75% with a 3-month cliff and a 6-month duration - Support batch operations for multiple investors to optimize gas costs during large distributions - Create vesting schedules through integration with SheVestingContract for controlled token release - Enable owner withdrawal of unsold tokens and management of private sale allocations - Provide single and batch investor onboarding with customizable token amounts per investor.	Contract achieves this functionality.
SheAllocation.sol - Distribute 1 billion SHE tokens across predefined categories (founders, developers, treasury, sales, community, reserves) - Create vesting schedules for founders and developers with a 6-month cliff and a 2-year linear vesting period	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Transfer immediate allocations to treasury, private sale, presale, and public sale contracts- Execute one-time token distribution with built-in protection against multiple distributions- Support batch vesting operations for multiple beneficiaries with different vesting parameters- Enable owner withdrawal of remaining tokens after distribution completion. | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the SafeHaven Exchange project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the SafeHaven Exchange project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] SheSaleContract#buyWithUSDT/withdrawFunds - Permanent revert when purchasing tokens for USDT

Description

The SheSaleContract contract contains a vulnerability in USDT token handling. The contract expects `USDT.transferFrom()/USDT.transfer` to return a boolean value, but the actual USDT contract implementation does not return any value for transfer functions, causing all USDT purchase transactions to revert regardless of success.

Vulnerability Details

The real USDT contract at [0xdAC17F958D2ee523a2206206994597C13D831ec7](https://etherscan.io/address/0xdAC17F958D2ee523a2206206994597C13D831ec7) does NOT return a boolean value from `transfer()` and `transferFrom()` functions

The contract code assumes USDT follows the standard ERC-20 interface, which requires boolean return values. These functions expect a return value but don't receive one; the transaction reverts

```
function buyWithUSDT(
    uint256 amountInUsdt
) external nonReentrant whenSaleActive {
    //...code

    // Transfer USDT from the buyer to the contract
    if (!USDT.transferFrom(msg.sender, address(this), amountInUsdt))
        revert USDTTransferFailed();
    //...code
}

function withdrawFunds() external onlyOwner {
    //...code
}
```



```

uint256 usdtBalance = USDT.balanceOf(address(this));
if (usdtBalance > 0) {
    if (!USDT.transfer(owner(), usdtBalance))
        revert USDTWithdrawalFailed();
    emit FundsWithdrawn(owner(), usdtBalance, "USDT");
}
//...code
}

```

Proof of Concept

The PoC demonstrates a revert when calling the function. Add and override the transfer and transferFrom functions so that they do not return a bool value.

The same thing happens when calling the withdrawFunds function.

```

// Helper function to set up a valid sale period
function _setupValidSalePeriod() internal {
    uint256 startTime = block.timestamp + 2 days;
    uint256 endTime = startTime + 14 days;

    vm.prank(owner);
    saleContract.setSalePeriod(startTime, endTime);
}

function test_H_01_TransferUSDT() public {
    _setupValidSalePeriod();
    vm.warp(saleContract.saleStartTime() + 1);
    vm.startPrank(buyer1);
    usdtToken.approve(address(saleContract), 10000 * 1e6);
    saleContract.buyWithUSDT(10000 * 1e6);
    vm.stopPrank();
}

```



```

forge test --mt test_H_01_TransferUSDT -vv

Ran 1 test for test/SheSaleContract.t.sol:SheSaleContractTest
[FAIL: USDTTransferFailed()] test_H_01_TransferUSDT() (gas: 144265)
Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 7.87ms (937.29µs CPU time)

Ran 1 test suite in 235.33ms (7.87ms CPU time): 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:
Encountered 1 failing test in test/SheSaleContract.t.sol:SheSaleContractTest
[FAIL: USDTTransferFailed()] test_H_01_TransferUSDT() (gas: 144265)

Encountered a total of 1 failing tests, 0 tests succeeded

```

Impact

All users attempting to purchase tokens with USDT will experience transaction failures. USDT is a major stablecoin; blocking this payment method significantly reduces accessibility.

All `withdrawFunds()` calls will revert when the USDT balance > 0 . Accordingly, all tokens, including ether, will be stuck in the contract.

Recommendation

Use OpenZeppelin's SafeERC20 library, which handles non-standard token implementations. Consider using this library in all contracts, not just for USDT.

Status

Resolved

Medium

[M-01] SheSaleContract#getEthPrice - Lack of validation of the latestRoundData return value

Description

The `getEthPrice` function in the `SheSaleContract` contract uses the `latestRoundData` function without validating whether the returned data is stale, potentially leading to the usage of outdated prices. Specifically, the function does not check the value of `updatedAt`, creating a vulnerability to stale prices.

Vulnerability Details

According to Chainlink's documentation, a `timestamp` of 0 means the round is not complete and should not be used. Without this check, the `getEthPrice` function could receive incorrect price data. Furthermore, the function does not check for an invalid `roundId`, which is 0.

```
function getEthPrice() public view returns (uint256) {
    // Fetch the latest ETH price in USD from Chainlink
    (
        ,
        /* uint80 roundID */ int256 price /*uint256 startedAt*/ /*uint256 timeStamp*/
/*uint80 answeredInRound*/,
        ,
        ,
    ) = ETHPriceFeed.latestRoundData();
    if (price <= 0) revert InvalidETHPrice();
    return uint256(price) * (10 ** (18 - ETHPriceFeed.decimals()));
}
```

Impact

The user will be forced to pay more ether than they have because the price feed returns a price higher than the market price. Or the user pays less ether because of an outdated price.

Recommendation

Add checks to `getEthPrice` to ensure that `updatedAt != 0`, that the feed data is still within an acceptable threshold (i.e., not stale), and that the current `roundId` is not 0.

Status

Resolved

[M-02] SheSaleContract#withdrawTokens - Centralization risk

Description

The `withdrawTokens()` function lacks proper access control during active sale periods, allowing the owner to withdraw all SHE tokens even when the sale is ongoing.

Vulnerability Details

Despite the comment stating *"Cannot be called during active sale period"*, there is no actual enforcement mechanism in the code. The function only has the `onlyOwner` modifier but lacks any check to prevent execution during an active sale.

```
/**
 * @dev Allows the owner to withdraw SHE tokens to their wallet.
 * Cannot be called during active sale period
 */
function withdrawTokens() external onlyOwner {
    uint256 balance = SHE.balanceOf(address(this));
    if (balance == 0) revert NoSHEToWithdraw();
    if (!SHE.transfer(owner(), balance)) revert SHEWithdrawalFailed();

    emit FundsWithdrawn(owner(), balance, "SHE");
}
```

Impact

The owner can withdraw all tokens during the active sale, preventing users from purchasing. This could lead to a loss of trust in the protocol.

Recommendation

Add a check that will call revert if sales are active.

Status

Resolved

[M-03] SheSaleContract#buyWithUSDC/buyWithETH/buyWithUSDT -

Frontrunning attack

Description

These functions in SheSaleContract lack slippage protection, allowing users to receive fewer tokens than expected due to price changes between transaction submission and execution.

Vulnerability Details

The function calculates the SHE token amount based on the current price without allowing users to specify minimum expected tokens:

```
function getSheFromUsd(uint256 usdtAmount) public view returns (uint256) {  
    return (usdtAmount * 1e18) / getTokenPrice(0);  
}
```

The contract has a variable MAX_PRICE_IMPACT, which is a protection against slippage and is equal to 10%, but is not used anywhere. This indicates that the protocol wanted to implement such protection.

```
// Add new state variables for slippage protection  
uint256 public constant MAX_PRICE_IMPACT = 10; // 10% maximum price impact
```

This issue is also relevant because the protocol will be deployed on the main Ethereum chain, where the mempool is public.

Proof of Concept

This PoC demonstrates how much less buyer2 will receive if buyer1's purchase transaction precedes theirs.

```

function test_PriceManipulation() public {
    assertEq(saleContract.userPurchases(buyer1), 0);
    assertEq(saleContract.userPurchases(buyer2), 0);
    //-----buyer1-----
    _setupValidSalePeriod();
    vm.warp(saleContract.saleStartTime() + 1);
    vm.startPrank(buyer1);
    usdcToken.approve(address(saleContract), 1 ether);
    saleContract.buyWithUSDC(5000 * 10 ** 6); // 1000 USDC
    vm.stopPrank();
    uint256 buyer1SheAmount = saleContract.userPurchases(buyer1);

    //-----buyer2-----
    vm.startPrank(buyer2);
    usdcToken.approve(address(saleContract), 1 ether);
    saleContract.buyWithUSDC(1000 * 10 ** 6); // 1000 USDC
    vm.stopPrank();
    uint256 buyer2SheAmount = saleContract.userPurchases(buyer2);

    console.log(buyer1SheAmount - buyer2SheAmount);
}

```

Test result:

```
forge test --mt test_PriceManipulation -vv
```

```
Ran 1 test for test/SheSaleContract.t.sol:SheSaleContractTest
```

```
[PASS] test_PriceManipulation() (gas: 568309)
```

```
Logs:
```

```
66831410859365116036018
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.72ms (1.97ms CPU time)
```

```
Ran 1 test suite in 250.26ms (8.72ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

```
cast --from-wei 66831410859365116036018
```

```
66831.410859365116036018
```



Impact

At the very beginning, these functions are vulnerable to frontrunning attacks. The expected number of tokens and the actual number will differ significantly.

Recommendation

Modify these functions to check for acceptable slippage.

Status

Resolved

[M-04] SheSaleContract#getTokenPrice - Inconsistency between NatSpec comments and function implementation / Lack of dynamic pricing

Description

A problem exists in the dynamic pricing mechanism of the SheSaleContract. The contract fails to implement proper dynamic pricing during token purchases, allowing users to bypass the intended price curve and purchase large amounts of tokens at artificially low prices.

Vulnerability Details

The comment states that this function has dynamic pricing, but the vulnerability arises from incorrect price calculation:

buyWithUSDT() calls getSheFromUsd(amountInUsdt) -> getSheFromUsd() calls getTokenPrice(0) (0 is passed in order to calculate the number of SHE tokens per 1 USDT). getTokenPrice(0) calculates the price based only on totalTokensSold + 0. Accordingly, the price changes only after the purchase is made. The price calculation ignores the impact of the current purchase on the price curve.

```
/**
 * @dev Returns the current price of 1 SHE token in USDT.
 * Formula:  $P(x) = 0.06 + 0.24 * (1 - \text{Math.pow}((1 - x / 1e8), 3))$ 
 * where x is the total number of tokens sold (dynamic pricing).
 */
function getTokenPrice(uint256 amount) public view returns (uint256) {
```



```

uint256 totalSold = (totalTokensSold + amount) / 1e18;

if (totalSold >= 1e8) {
    return INITIAL_PRICE + PRICE_RANGE;
}

uint256 normalizedSoldRatio = 1e8 - totalSold;
uint256 ratioCubed = (normalizedSoldRatio *
    normalizedSoldRatio *
    normalizedSoldRatio) / 1e16;

return INITIAL_PRICE + (PRICE_RANGE * (1e8 - ratioCubed)) / 1e8;
}
//...code
function getSheFromUsd(uint256 usdtAmount) public view returns (uint256) {
    return (usdtAmount * 1e18) / getTokenPrice(0);
}

//...code
function buyWithUSDT(
    uint256 amountInUsdt
) external nonReentrant whenSaleActive {
    if (amountInUsdt == 0) revert InvalidAmount();

    uint256 sheAmount = getSheFromUsd(amountInUsdt);

    //...code
    // Update total tokens sold and funds raised
    totalTokensSold += sheAmount;
    totalFundsRaised += amountInUsdt;

    //...code
}

```

In this case, whales can buy a large amount of SHE tokens at INITIAL_PRICE. The only limit for them will be 1 hour before each purchase, but this can be avoided by using different addresses and MAX_PURCHASE_AMOUNT per transaction. This can also be avoided by using different addresses and getting ahead of other users' transactions.

Proof of Concept

This PoC demonstrates the different prices when purchasing SHE tokens for the same amount of USDT.

```
function test_NonDynamicPrice() public {
    _setupValidSalePeriod();
    assertEq(saleContract.getTokenPrice(0), saleContract.INITIAL_PRICE());
    vm.warp(saleContract.saleStartTime() + 1);
    vm.startPrank(buyer1);
    usdcToken.approve(address(saleContract), 1 ether);
    saleContract.buyWithUSDC(10_000 * 10 ** 6); // 10000 USDC

    uint256 onePurchase = saleContract.userPurchases(buyer1);
    vm.stopPrank();
    assertGt(saleContract.getTokenPrice(0), saleContract.INITIAL_PRICE()); // The
price has increased.

    //-----Updating the status at the time of deployment-----
    vm.revertTo(snapshot);
    assertEq(saleContract.getTokenPrice(0), saleContract.INITIAL_PRICE()); // Because
of the reset, the price went back to INITIAL_PRICE.
    _setupValidSalePeriod();
    vm.warp(saleContract.saleStartTime() + 1);
    vm.startPrank(buyer1);
    usdcToken.approve(address(saleContract), 1 ether);
    saleContract.buyWithUSDC(5_000 * 10 ** 6); // 5000 USDC
    vm.stopPrank();
    uint256 buyer1Purchase1 = saleContract.userPurchases(buyer1);

    vm.startPrank(buyer2); // We use the second buyer because of the cooldown.
    usdcToken.approve(address(saleContract), 1 ether);
    saleContract.buyWithUSDC(5_000 * 10 ** 6); // 5000 USDC
    vm.stopPrank();
    uint256 buyer1Purchase2 = saleContract.userPurchases(buyer2);

    console.log(onePurchase - (buyer1Purchase1 + buyer1Purchase2));
}
```


Test result:

```
forge test --mt test_NonDynamicPrice -vv

Ran 1 test for test/SheSaleContract.t.sol:SheSaleContractTest
[PASS] test_NonDynamicPrice() (gas: 905304)
Logs:
  823720963492246846758

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.87ms (1.70ms CPU time)

Ran 1 test suite in 237.83ms (8.87ms CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)

cast --from-wei 823720963492246846758
823.720963492246846758
```

Impact

Dynamic pricing is not dynamic. This leads to the contract initially receiving less USD for the purchase of tokens. It also gives an advantage to the very first buyers, as they will be buying at an artificially low price.

Recommendation

Implement the correct calculation of dynamic pricing or fix the NatSpec comment.

Status

Resolved

Low

[L-01] Contracts - No checks for address 0

Description

Many functions do not check for the address 0. This can lead to unexpected consequences in the future.

Recommendation

Consider adding a check for the address 0.

Status

Resolved

[L-02] SheSaleContract - Incorrect comparison

Description

There is a logical inconsistency in the time boundary checks within the `_checkAndEndSale()` function and the `whenSaleActive` modifier regarding the sale end time validation.

```
modifier whenSaleActive() {  
    if (!saleActive) revert SaleNotActive();  
    if (block.timestamp < saleStartTime) revert SaleNotStarted();  
=>    if (block.timestamp > saleEndTime) revert SaleEnded();  
    if (SHE.balanceOf(address(this)) == 0)  
        revert NotEnoughTokensAvailable();  
    -;  
}  
  
function _checkAndEndSale() internal {  
    if (block.timestamp > saleEndTime) {  
        saleActive = false;  
        //...code  
    }  
}
```

When `block.timestamp == saleEndTime` (exactly at the sale end time):

The `whenSaleActive` modifier allows the transaction to proceed (since `block.timestamp > saleEndTime` is false). The `_checkAndEndSale()` function does not terminate the sale (since `block.timestamp > saleEndTime` is false). This creates a window where users can still purchase tokens exactly at the intended sale end time.

The `setSaleStatus` function contains a similar comparison, but here the comparison is not strict.

```
function setSaleStatus(bool _active) external onlyOwner {  
    //...code  
  
    // Prevent reactivation if sale has ended  
    if (!_active && block.timestamp >= saleEndTime) {  
        revert SaleAlreadyEnded();  
    }  
    //...code  
}
```

Recommendation

Change strict comparison to non-strict comparison

Status

Resolved

Gas

[G-01] contracts - Don't initialize to default values

Description

Contracts such as `SheVestingContract`, `SheSaleContract`, and `ShePrivateSaleContract` contain declarations of variables with their default values. For `uint/int` it is 0, for `bool` it is `false`. It requires more gas.

Proof of Concept

This PoC demonstrates the deployment of the `SheSaleContract` contract with and without variable declarations.

Run this test, then comment out the declaration of the corresponding variables in the constructor, and run the test again.

```
function test_G_01_Deploy() public {
    vm.startPrank(owner);
    uint256 gasBefore = gasleft();
    SheSaleContract saleContract = new SheSaleContract(
        address(sheToken),
        address(usdtToken),
        address(usdcToken),
        address(ethPriceFeed),
        address(vestingContract)
    );
    uint256 gasAfter = gasleft();

    vm.stopPrank();

    assertEq(saleContract.saleActive(), false);
    assertEq(saleContract.saleStartTime(), 0);
    assertEq(saleContract.saleEndTime(), 0);
}
// gasWithDeclaration 2_041_750
// gasWithoutDeclaration 2_035_109
```

Recommendation

Consider removing the value assignment to zero and just declaring the variables.

Status

Resolved

[G-02] SheVestingContract#setTokenContract - Dead function

Description

This function is dead because it cannot be called. When the contract is deployed, it sets the `tokenSet` variable to true, and when this function is called, it first checks that this variable is not set. Because of this, calling this function will always revert.

```
constructor(address tokenAddress) {
    token = IERC20(tokenAddress);
    owner = msg.sender;
    tokenSet = true;
}

function setTokenContract(address tokenAddress) external onlyOwner {
    if (tokenSet) revert TokenAlreadySet();
    token = IERC20(tokenAddress);
    tokenSet = true;
}
```

Also note the constant variable `MAX_SALE_AMOUNT`. It is declared but not used anywhere. And this line in the `buyWithETH` function:

```
function buyWithETH() external payable nonReentrant whenSaleActive {
    //...code
    // Refund any excess ETH sent by the user
    if (ethAmount > actualCostInEth) {
        uint256 refundAmount = ethAmount - actualCostInEth;
        => if (refundAmount == 0) revert InvalidRefundAmount();
    }
    //...code
```

```
}
```

This line does not make sense because `refundAmount` will never be equal to `0`, since `ethAmount > actualCostInEth`.

Recommendation

Remove dead code

Status

Resolved

[G-03] SheAllocation - Use immutable variables

Description

The `vesting` variable in this contract is a `public` variable. However, this variable does not change after the constructor, and the corresponding `SheVestingContract` contract is not upgradeable, so declaring this variable as `immutable` helps save gas when calling functions with this variable.

Recommendation

Consider changing the variable to `immutable`. This will increase gas consumption during deployment, but will save on subsequent function calls.

Status

Resolved

QA

[Q-01] SheVestingContract#onlyControllers - Inappropriate modifier name

Description

The `onlyControllers` modifier before calling the function will check that `msg.sender` is either a controller or the contract owner, but the name of the modifier suggests that only controllers can call functions with the corresponding modifier. This is confusing.

Recommendation

Consider changing the name of the modifier to `onlyControllersAndOwner`.

Status

Resolved

[Q-02] Contracts - Order of functions

Description

For readability, you need to follow the correct order of contract construction. For example, the `SheVestingContract` contract lacks a construction structure. First come the functions, then the constructor, then the functions again. This makes the code difficult to read.

```
function removeControllers(  
    address[] calldata oldControllers  
) external onlyOwner {  
    //...code  
}  
  
constructor(address tokenAddress) {  
    //...code  
}  
  
function setTokenContract(address tokenAddress) external onlyOwner {  
    //...code  
}
```

```
}
```

Recommendation

Consider changing the structure of the contract in accordance with a clear procedure.

Status

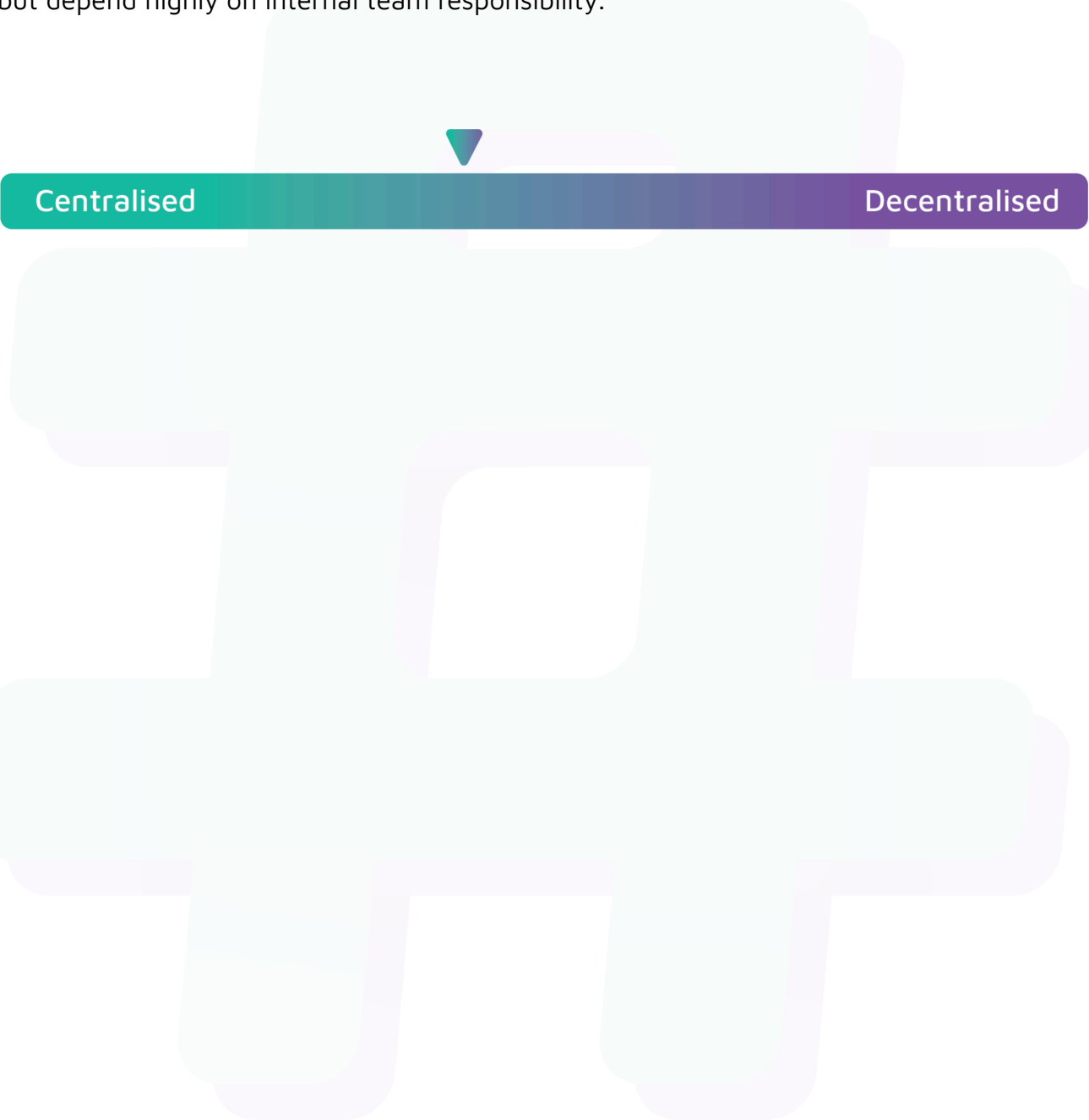
Resolved



Centralisation

The SafeHaven Exchange project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the SafeHaven Exchange project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.